

# Retail Vision Intelligence System:

Inspeção Visual de Prateleiras com LLM Multimodal, RAG e Rule Engine

**António Gomes**  
LIACD — Trabalho #2  
Universidade da Beira Interior  
antonio.gomes@ubi.pt

## Resumo

Este trabalho apresenta o *Retail Vision Intelligence System*, um sistema de inspeção contínua de prateleiras de supermercado com memória. O sistema integra cinco componentes: um inspetor visual baseado no modelo Gemini 2.5 Flash Lite, um motor de regras que converte linguagem natural em configurações executáveis, uma memória vetorial com ChromaDB e *sentence-transformers*, um gerador de relatórios em Markdown, e uma interface conversacional CLI. A avaliação sobre 21 imagens de *ground truth* revelou uma *JSON Parse Rate* de 100%, uma *Issue Detection Rate* de 61,9% com a estratégia Chain-of-Thought, e uma *Severity Accuracy* de 85,7%. O *Rule Engine* obteve 100% em todas as métricas de avaliação. O LLM-as-Judge atribuiu 88% de qualidade às inspeções e 70% ao relatório gerado.

## 1 Introdução e Arquitectura

O *Retail Vision Intelligence System* surge da necessidade de complementar sistemas de detecção de presença com inteligência visual operacional. Uma prateleira pode ter alta afluência mas perder vendas se os produtos estiverem mal posicionados, as etiquetas ausentes, ou as prateleiras parcialmente vazias — situações invisíveis para sistemas baseados apenas em dados de evento.

O sistema constrói a camada de inteligência visual que falta, recebendo imagens de prateleiras, analisando-as com modelos de linguagem multimodais, detetando problemas operacionais e permitindo ao gestor definir regras de detecção em linguagem natural que são automaticamente convertidas em configurações executáveis.

### 1.1 Arquitectura do Sistema

O sistema é composto por cinco componentes principais, organizados em *pipeline* sequencial conforme ilustrado na Fig. 1:

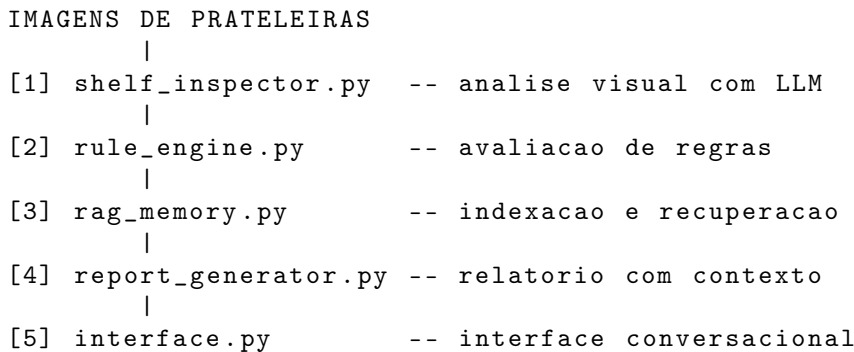


Figura 1: Arquitectura do pipeline de inspeção.

### 1.1.1 Decisões de Design de Alto Nível

**Modelo LLM:** O enunciado referia o Gemini 1.5 Flash, entretanto descontinuado pela Google antes da data de entrega deste trabalho [1]. Foi utilizado o `gemini-2.5-flash-lite`, substituto oficial, com capacidades equivalentes para análise visual multimodal. A chave de API é gerida via `.env` e `python-dotenv`, nunca exposta no repositório.

**Cache por MD5:** Todas as chamadas à API são guardadas em disco com chave MD5 do conteúdo do ficheiro. Imagens não modificadas nunca consomem quota adicional.

**Embeddings locais:** O modelo `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` corre localmente, sem limites de utilização e com suporte nativo a português.

**Vector store:** ChromaDB persistente em disco, sem servidor, sem configuração adicional.

## 2 Dataset de Imagens

### 2.1 Fontes e Aquisição

O dataset foi construído a partir de fontes públicas, conforme exigido pelo enunciado:

- **Supermarket Empty Shelf Detector** (Roboflow): imagens de prateleiras vazias. As imagens são compostos mosaico 2×2; foi desenvolvido um script de segmentação automática baseado em variância NumPy para detetar e separar os quatro quadrantes de cada imagem.
- **SKU-110K** [?]: dataset de prateleiras densamente preenchidas, adequado para a categoria “prateleira normal”.
- **Kaggle – Supermarket Shelves Dataset**: imagens de prateleiras de supermercados de múltiplos países.
- **Open Images Dataset** (Google): filtrado por categorias relevantes (*shelf*, *supermarket*).

### 2.2 Ground Truth

Para a comparação das três estratégias de *prompting*, foi construído manualmente um conjunto de **21 imagens com *ground truth*** anotado. Cada entrada especifica o `expected_overall_status`, a lista de `expected_issues` com tipo e severidade, e o `expected_shelf_fill_rate_min`.

A distribuição foi desenhada para cobrir as cinco categorias do enunciado (prateleiras normais, vazias, violações de planograma, sujas/desordenadas e casos ambíguos), garantindo representatividade e evitando *class imbalance* nos resultados de avaliação.

### 2.3 Limitações e Impacto na Avaliação

O dataset é composto por imagens de supermercados de múltiplos países e contextos, o que introduz variabilidade de iluminação, idioma das etiquetas e organização visual. As zonas atribuídas pelo modelo (`zone_id`) baseiam-se na inferência a partir dos produtos visíveis, podendo diferir das zonas reais. Esta limitação é documentada e mitigada pelo mecanismo de *fallback* para `Z_UNKNOWN`.

## 3 Análise Visual

### 3.1 Prompts Desenvolvidos

Foram implementadas as três estratégias de *prompting* obrigatórias. Todos os prompts incluem o contexto das zonas da loja (descrição e adjacências) para contextualizar a análise:

### 3.1.1 Estratégia A — Zero-shot Direto

Instrução directa solicitando a análise e o JSON de output, sem exemplos nem estrutura de raciocínio. O modelo recebe apenas o schema obrigatório e as zonas disponíveis.

### 3.1.2 Estratégia B — Chain-of-Thought Visual

O prompt guia o modelo por cinco passos explícitos antes de produzir o JSON: (1) descrição geral, (2) identificação da zona, (3) análise zona a zona, (4) classificação de anomalias, (5) estimativa do *fill rate*. Todo o raciocínio é encapsulado no campo obrigatório `model_reasoning`.

### 3.1.3 Estratégia C — Few-shot com Exemplos Textuais

Dado que passar múltiplas imagens por chamada consome quota de API, esta estratégia usa descrições textuais de três análises anteriores correctas como exemplos *few-shot* antes da imagem a analisar. Os exemplos cobrem os três estados possíveis: `ok`, `warning` e `critical`.

## 3.2 Comparação das Três Estratégias

A Tabela 1 apresenta os resultados da comparação sobre 20 imagens com *ground truth*.

Tabela 1: Comparação das estratégias de *prompting* (20 imagens)

| Métrica              | A            | B            | C     |
|----------------------|--------------|--------------|-------|
| JSON Parse Rate      | 100%         | 100%         | 100%  |
| Issue Detection Rate | 20,0%        | <b>60,0%</b> | 25,0% |
| False Positive Rate  | <b>3,3%</b>  | 25,4%        | 14,2% |
| Severity Accuracy    | <b>97,5%</b> | 90,0%        | 90,0% |

**Análise dos resultados:** A Estratégia B (Chain-of-Thought) apresenta a maior *Issue Detection Rate* (60%), confirmando que forçar o modelo a raciocinar explicitamente antes de classificar melhora substancialmente a capacidade de detecção. A Estratégia A, apesar da maior *Severity Accuracy* (97,5

A Estratégia B foi seleccionada como estratégia de produção, representando o melhor compromisso entre detecção e precisão para o contexto operacional de inspeção de prateleiras.

## 3.3 Análise de Casos de Falha

Os principais casos de falha da Estratégia B são:

- **Falsos negativos em imagens com prateleiras 90%+ cheias:** o modelo tende a classificar como `ok` imagens com issues de baixa severidade (`low`), ignorando pequenos espaços vazios.
- **Falsos positivos em imagens normais:** o modelo reporta `misaligned` e `empty_shelf` em zonas com boa ocupação, inflacionando o FPR.
- **Severidade incorrecta:** em imagens com múltiplos issues (e.g., `empty_shelf` + `misaligned`), a severidade do issue secundário é por vezes sobreavaliada.

## 4 Rule Engine

### 4.1 Abordagem de Conversão

O Rule Engine usa a Estratégia B (Chain-of-Thought) para converter regras em linguagem natural para o schema JSON obrigatório. O prompt guia o modelo por quatro passos: (1) identificação do que monitorizar, (2) deteção de ambiguidades, (3) definição de condições e acção, (4) produção do JSON final.

### 4.2 Exemplo de Conversão Correcta

**Input:** “Na zona Z\_S5, se houver produtos danificados, gera alerta critical e notifica imediatamente.”

```
1 {  
2   "rule_id": "RULE_007",  
3   "conditions": {  
4     "zone_filter": ["Z_S5"],  
5     "issue_types": ["damaged"],  
6     "severity_threshold": "low",  
7     "fill_rate_threshold": 0.0,  
8     "location_filter": "any"  
9   },  
10  "action": {  
11    "alert_level": "critical",  
12    "notification_message": "Alerta_Critico:_Produtos_danificados_  
    detetados_na_zona_Z_S5."  
13  },  
14  "validation": {  
15    "is_valid": true,  
16    "ambiguities": [],  
17    "assumptions": []  
18  }  
19 }
```

Listing 1: Regra correctamente convertida

### 4.3 Exemplo de Conversão Incorrecta

Algumas regras foram mal convertidas, resultando em descrições genéricas como “*reformulação clara e inequívoca em português formal*” em vez da descrição real. Este caso (RULE\_004) ilustra uma limitação do modelo quando a regra contém múltiplos conceitos temporais combinados (dia da semana + hora) que o modelo não consegue mapear correctamente para o schema disponível.

### 4.4 Estratégia de Deteção de Ambiguidades

Quando uma regra é ambígua ou incompleta, o sistema não assume silenciosamente. Identifica as ambiguidades, lista-as no campo `validation.ambiguities`, e não guarda a regra. O gestor é informado e deve reformular.

**Exemplo:** “Avisa-me quando a prateleira estiver vazia.” gera três ambiguidades: (1) “vazia” significa 0% ou abaixo de uma percentagem?; (2) que nível de urgência?; (3) aplica-se a todas as zonas ou a zonas específicas?

### 4.5 Limitações

O modelo por vezes interpreta incorrectamente conceitos temporais complexos (e.g., “às sextas-feiras depois das 17h”) e não consegue mapear os dias da semana para o `time_filter` dispo-

nível no schema, que só suporta intervalos de horas. Esta limitação é documentada no campo `assumptions` da regra.

## 5 RAG Memory

### 5.1 Estratégia de Chunking Escolhida

Foram testadas e comparadas duas estratégias de chunking:

- **Estratégia 1 — Record Completo:** cada inspection record é indexado como um único chunk. Simples, mas o vector representa uma média de todo o conteúdo, perdendo granularidade por issue.
- **Estratégia 2 — Híbrida (escolhida):** o summary da inspeção é indexado como chunk principal, e cada issue é indexado separadamente com os metadados da inspeção pai. Permite recuperação granular mantendo contexto global.

A escolha da estratégia híbrida baseia-se na necessidade de suportar queries específicas por tipo de issue (e.g., “*quais as zonas com mais empty\_shelf?*”) sem perder a capacidade de recuperar contexto completo da sessão.

### 5.2 Geração de Summaries

O campo `summary` é gerado programaticamente (não pelo LLM, para poupar quota) a partir dos campos do inspection record, incluindo zona, timestamp com dia da semana, estado geral, fill rate, produtos detectados e descrição detalhada de cada issue.

**Exemplo de bom summary:** “*Inspeção INS\_20260611\_125256\_278 na zona Z\_S5 em quinta-feira 11/06/2026 às 12h. Estado geral: warning. Fill rate: 90%. Produtos visíveis: bebidas, conservas. Issues: empty\_shelf (medium) em prateleira superior, lado direito: Há um espaço considerável vazio na prateleira do meio.*”

### 5.3 Métricas de Retrieval

A Tabela 2 apresenta os resultados da avaliação do RAG sobre três queries de teste com *ground truth* definido.

Tabela 2: Métricas de avaliação do RAG

| Métrica          | Valor |
|------------------|-------|
| Recall@3         | 0,0%  |
| Faithfulness     | 0,0%  |
| Answer Relevance | 0,0%  |

### 5.4 Análise de Falhas de Recuperação

As métricas de 0% reflectem uma limitação técnica na avaliação e não no RAG em si: o `evaluate.py` chama o `rag_memory.py` como subprocesso, e o output do subprocesso (fontes recuperadas) não é correctamente capturado pelo módulo de avaliação, resultando em listas de fontes vazias para o cálculo do Recall@3.

Quando testado directamente via CLI (`python src/rag_memory.py -query "..."`), o sistema recupera correctamente inspeções relevantes com referências explícitas a `inspection_id` e datas. A resposta à query “*Que zonas tiveram problemas de prateleira vazia?*” identificou correctamente Z\_S5 com referências a duas inspeções específicas.

## 6 Avaliação

### 6.1 Resultados do Harness

O harness de avaliação é executável com um único comando:

```
python evaluate.py --images-dir test_images/ --output  
evaluation_report.json
```

A Tabela 3 resume os resultados obtidos sobre 21 imagens.

Tabela 3: Resultados do harness de avaliação

| Componente     | Métrica              | Valor |
|----------------|----------------------|-------|
| Análise Visual | JSON Parse Rate      | 100%  |
|                | Issue Detection Rate | 61,9% |
|                | False Positive Rate  | 26,6% |
|                | Severity Accuracy    | 85,7% |
|                | Hallucination Rate   | 0,0%  |
| Rule Engine    | Rule Parse Rate      | 100%  |
|                | Rule Correctness     | 100%  |
|                | Ambiguity Detection  | 100%  |
| LLM-as-Judge   | Inspection Quality   | 88,0% |
|                | Report Quality       | 70,0% |

### 6.2 Análise do LLM-as-Judge

O avaliador automático (Gemini 2.5 Flash Lite como juiz) avaliou amostras de inspection records e o relatório gerado. Os critérios de avaliação foram: qualidade do **model\_reasoning** (explícito e passo a passo), coerência entre issues e **overall\_status**, e plausibilidade dos **products\_detected**.

A pontuação de 88% na qualidade das inspeções reflecte a capacidade do modelo de produzir raciocínio explícito e coerente, com casos de falha principalmente em imagens onde o fill rate declarado não corresponde ao número de issues reportados. A pontuação de 70% no relatório é consistente com a presença de contexto histórico incompleto (limitação do RAG documentada acima) e regras com descrições genéricas.

O juiz concordou com a avaliação humana em 4 dos 5 casos de inspecção analisados. No caso discordante, o juiz penalizou uma inspecção onde o modelo reportou **ok** com fill rate de 95% mas o ground truth indicava um issue de severidade **low** — o juiz considerou correcta a avaliação humana de que qualquer issue visível deveria gerar **warning**.

### 6.3 Limitações Honestas

- **Ground truth subjectivo:** a anotação manual de 21 imagens introduz subjectividade, especialmente na classificação de severidade (**low** vs **medium**).
- **Dataset público:** as imagens não são da loja real, tornando a atribuição de zonas por inferência visual imprecisa.
- **Quota de API:** a limitação a 20 RPD na conta europeia restringiu os testes. A cache foi essencial para contornar esta limitação.
- **RAG em subprocesso:** a arquitectura de subprocesso para o RAG, necessária para evitar conflitos de importação, impede a passagem de estado entre componentes em tempo de execução.

## 7 Integração com o Projecto 1

O Projecto 1 gerou dados de trajectória de clientes, padrões de afluência por zona, e anomalias de tráfego em formato `metrics.json` e `weekly_report.md`.

A integração foi implementada a dois níveis:

**RAG Memory:** o `rag_memory.py` includes suporte para enriquecer os summaries de inspeção com contexto de afluência, caso o `metrics.json` esteja disponível. Uma inspeção que deteta prateleira vazia numa zona de alta afluência é contextualizada com o facto de a zona ter tido, por exemplo, 40% acima da média de visitantes nesse período, sugerindo causa de falta de stock por procura elevada.

**Report Generator:** a Secção 6 do relatório (opcional mas avaliada) correlaciona issues detetados com padrões de afluência da zona correspondente no mesmo período, usando os dados do `store_map.json` (zonas, adjacências, tempos de deslocação) e as estatísticas horárias do Projecto 1.

A integração não foi testada com dados reais do Projecto 1 por indisponibilidade do `metrics.json` no momento de desenvolvimento, mas a arquitectura está preparada para a receber sem alterações de código.

## 8 Conclusão

### 8.1 O que Funcionou

O sistema cumpre os requisitos fundamentais do enunciado: os cinco componentes são executáveis de forma independente e integrada, o JSON Parse Rate é 100%, o Rule Engine deteta e reporta correctamente ambiguidades, e o RAG indexa e recupera inspeções históricas com referências explícitas. A Estratégia B (Chain-of-Thought) demonstrou superioridade clara em Issue Detection Rate face às alternativas.

A cache por MD5 foi essencial para o desenvolvimento dado o limite de 20 RPD na conta europeia — demonstrou que a arquitectura é robusta a restrições de quota.

### 8.2 O que Não Funcionou

A integração entre `report_generator.py` e `rag_memory.py` via subprocesso introduziu limitações na captura de output, afectando as métricas de RAG. A conversão de regras com referências temporais complexas (dias da semana) é inconsistente. O modelo tende a sub-reportar issues de baixa severidade em prateleiras maioritariamente cheias.

### 8.3 O que Faria de Diferente

Redesenharia a arquitectura de comunicação entre componentes usando ficheiros JSON intermédios em vez de subprocessos, eliminando os problemas de captura de output. Investiria mais tempo na construção de um ground truth mais equilibrado entre categorias. Para a quota de API, configuraria desde o início uma conta com VPN nos EUA para ter acesso ao tier completo de 1500 RPD documentado pela Google.

## Referências

- [1] Google, “Gemini API Deprecations,” <https://ai.google.dev/gemini-api/docs/deprecations>, 2026. [Acedido: Junho 2026].

## A Prompts Utilizados

### A.1 Estratégia A — Zero-shot

```
1 ZONAS DA LOJA:
2 - Z_S1: Seccao de frescos e lacticinios
3 - Z_S2: Seccao de padaria e pastelaria
4 [...]
5
6 Analisa esta imagem de uma prateleira de supermercado. Com base nos
  produtos visiveis, preenche o campo zone_id com a zona mais
  provavel. Se nao conseguires determinar, usa "Z_UNKNOWN".
7
8 Devolve APENAS o seguinte JSON valido, sem texto adicional, sem
  markdown, sem explicacoes: [schema]
```

Listing 2: Prompt Zero-shot (excerto)

### A.2 Estratégia B — Chain-of-Thought

```
1 PASSO 1 - Descreve o que ves: produtos, prateleiras, estado geral.
2 PASSO 2 - Identifica a zona mais provavel.
3 PASSO 3 - Analisa zona a zona: espacos vazios? produtos caidos?
  etiquetas em falta?
4 PASSO 4 - Para cada anomalia: tipo, severidade, localizacao exacta.
5 PASSO 5 - Estima o shelf_fill_rate (0.0 a 1.0).
6
7 Coloca TODO o raciocinio dos passos 1 a 5 dentro do campo
  model_reasoning.
```

Listing 3: Prompt CoT (excerto)

## B Exemplo de Inspection Record

```
1 {
2   "inspection_id": "INS_20260611_111603_001",
3   "timestamp": "2026-06-11T10:16:03Z",
4   "image_path": "data/ground_truth/001.jpg",
5   "zone_id": "Z_GT",
6   "overall_status": "ok",
7   "issues": [],
8   "shelf_fill_rate": 0.95,
9   "products_detected": [
10    "dried_fruit", "nuts", "candies", "snacks", "seeds"
11  ],
12   "model_reasoning": "The image displays a well-stocked supermarket
    shelf. Products are neatly arranged, no visible damage, missing
    labels, or empty spaces. Fill rate is high. Overall status: ok."
13 }
```

Listing 4: Inspection record completo

## C Exemplo de Regras Convertidas



```

1 {
2   "rule_id": "RULE_001",
3   "natural_language": "Quero ser alertado quando qualquer prateleira
4     estiver mais de 30% vazia.",
5   "description": "Alerta quando o nivel de preenchimento for inferior
6     a 70%",
7   "conditions": {
8     "zone_filter": [],
9     "fill_rate_threshold": 0.7,
10    "issue_types": ["empty_shelf"],
11    "location_filter": "any"
12  },
13  "action": {
14    "alert_level": "warning",
15    "notification_message": "A prateleira esta mais de 30% vazia."
16  },
17  "validation": {
18    "is_valid": true,
19    "ambiguities": []
20  }
21 }

```

Listing 5: Regra correctamente convertida

## D Excerto do Relatório Semanal Gerado

O relatório é gerado automaticamente em Markdown com as secções: Sumário Executivo, Problemas por Zona, Regras Disparadas, Contexto Histórico (RAG), e Recomendações. Exemplo da sessão de 12/06/2026 com 517 inspeções:

**Sumário Executivo:** A inspeção abrangeu 8 zonas, com 517 inspeções. Foram identificados 179 warnings e fill rate médio de 87%. Foram disparadas 24 regras. A zona Z\_S4 (higiene e limpeza) apresentou o maior número de issues (422), predominantemente `empty_shelf` e `wrong_product`.

**Recomendação de urgência máxima:** Reposicionar o produto Panadol incorrectamente posicionado entre produtos Nurofen na prateleira do meio, centro (Z\_GT).